

腾讯文档 前端开发

一面（技术基础） 1-10

1. 谈谈你对CSS 盒模型的理解，以及 box-sizing的作用。
2. 说一下 JavaScript 的事件循环机制（Event Loop）。
3. 手写实现一个 Promise.all 方法。
4. 解释一下 JS 中的原型链以及如何实现继承。
5. 谈谈 HTTP 缓存策略（强缓存与协商缓存）。
6. 如何实现一个深拷贝（Deep Clone）？
7. Flex 布局中 flex:1代表什么意思？
8. 什么是闭包？它有哪些应用场景和副作用？
9. 数组去重有哪些实现方式？
10. 算法题：将一个扁平的数组转换为树形结构。

二面（深入原理与项目） · 1-10

1. 谈谈 React Fiber 架构解决了什么问题？它的工作原理是什么？
2. React 中的 Diff 算法是如何优化的？
3. 你们项目里是如何进行性能优化的？有哪些量化指标？
4. Webpack 的构建流程是怎样的？如何提高构建速度？
5. 谈谈 Canvas 和 SVG 的区别，在文档场景下如何选择？
6. 协同编辑中的 OT 算法或 CRDT 算法了解吗？
7. 什么是微前端？你们是如何处理微前端应用间的通信和样式隔离的？
8. 谈谈前端安全，如何防范 XSS 和 CSRF 攻击？
9. 虚拟列表（Virtual List）的实现原理是什么？
10. 介绍一下你负责过的最难的项目，难点在哪里？你是如何解决的？

三面（架构与综合能力） 1-8

1. 如果让你从零设计一个前端监控系统，你会考虑哪些模块？
2. 面对一个大型老旧项目，你会如何制定技术升级或重构计划？
3. 谈谈你对前端工程化的理解，它包含哪些内容？
4. 在多人协作的复杂项目中，如何保证代码质量和风格统一？

5. 跨端技术（RN,Flutter, H5）你如何看待？在什么场景下选哪种？
6. 你最近在关注哪些前端新技术？它们解决了什么问题？
7. 聊聊你在项目中遇到的最大挫折或失败的经历。
8. 你对腾讯文档前端团队有什么了解？你觉得自己为什么适合这个岗位？

一面（技术基础）

1. 谈谈你对 CSS 盒模型的理解，以及 box-sizing 的作用。

难度：★

考点：CSS 基础布局

答案要点：

盒模型是浏览器对 HTML 元素进行布局的基础，它规定了元素如何计算宽度和高度。

- 标准盒模型：width 仅包含 content，不包含 padding 和 border。
- 怪异盒模型（IE 盒模型）：width 包含 content、padding 和 border。
- box-sizing 属性：通过 content-box 或 border-box 进行切换，现代开发多用 border-box。
- 影响因素：外边距折叠（Margin Collapsing）通常发生在垂直方向的相邻块级元素间。

常见坑：

- 忽略了 padding 和 border 会撑开标准盒模型的实际宽度。
- 混淆了行内元素和块级元素在盒模型表现上的差异。

追问：

- 什么是 BFC？如何触发 BFC？

2. 说一下 JavaScript 的事件循环机制（Event Loop）。

难度：★★

考点：JS 异步执行机制

答案要点：

事件循环是 JS 处理异步任务的核心机制，通过任务队列的调度实现非阻塞 I/O。

- 执行栈与任务队列：先执行同步代码，再处理异步回调。
- 微任务（Microtask）：Promise.then、MutationObserver、process.nextTick。
- 宏任务（Macrotask）：setTimeout、setInterval、I/O、UI 渲染。
- 执行顺序：执行一个宏任务 -> 清空所有微任务 -> 渲染页面 -> 下一个宏任务。

常见坑：

- 错误地认为 setTimeout(0) 会立即执行。
- 不清楚 await 后面的代码会被包装成微任务。

追问：

- Node.js 的事件循环和浏览器端有什么区别？

3. 手写实现一个 Promise.all 方法。

难度：★★

考点：异步编程、手写代码能力

答案要点：

Promise.all 接收一个 Promise 数组，只有当所有 Promise 都成功时才返回成功结果数组，只要有一个失败就立即返回失败。

- 输入处理：需要处理非 Promise 类型的元素。
- 计数器机制：维护一个已完成的任务计数，当计数等于数组长度时 resolve。
- 顺序保证：返回的结果数组顺序必须与输入数组顺序一致。
- 错误处理：任何一个任务 reject 都要直接调用 reject。

代码块

```
1  function promiseAll(promises) {
2    return new Promise((resolve, reject) => {
3      const result = [];
4      let count = 0;
5      promises.forEach((p, index) => {
6        Promise.resolve(p).then(res => {
7          result[index] = res;
8          count++;
9          if (count === promises.length) resolve(result);
10         }, err => reject(err));
11      });
12    });
13  }
```

常见坑：

- 结果数组顺序错乱（直接用 push 而不是通过索引赋值）。
- 忘记处理输入为空数组的情况。

追问：

- Promise.allSettled 和 Promise.race 的区别是什么？

4. 解释一下 JS 中的原型链以及如何实现继承。

难度：★★

考点：面向对象、原型基础

答案要点：

原型链是 JS 实现对象间属性共享和继承的机制，每个对象都有一个指向其构造函数原型的内部链接。

- 原型对象：每个函数都有 prototype 属性，每个对象都有 **proto** 属性。
- 查找机制：访问属性时，若当前对象没有，则顺着 **proto** 向上查找直到 null。
- 继承方式：ES6 的 class 继承（extends）是主流，底层仍是原型链。
- 构造函数继承与组合继承：通过 call 借用构造函数，通过原型链继承方法。

常见坑：

- 忘记修复 constructor 的指向。
- 对引用类型属性在原型链上的共享问题理解不深。

追问：

- class 语法糖和 ES5 构造函数继承在底层有什么细微区别？

5. 谈谈 HTTP 缓存策略（强缓存与协商缓存）。

难度：★★

考点：网络基础、性能优化

答案要点：

HTTP 缓存通过减少重复的网络请求来提升页面加载速度，分为强缓存和协商缓存两个阶段。

- 强缓存：通过 Cache-Control（max-age）或 Expires 判断，命中则不发请求。
- 协商缓存：强缓存失效后，携带 If-None-Match (Etag) 或 If-Modified-Since 请求服务器。
- 状态码：强缓存命中返回 200 (from cache)，协商缓存命中返回 304 (Not Modified)。
- 优先级：Cache-Control 优先级高于 Expires，Etag 优先级高于 Last-Modified。

常见坑：

- 认为 304 也会传输完整的响应体。
- 不清楚 Etag 解决的是 Last-Modified 时间精度不够的问题。

追问：

- 为什么有了 Last-Modified 还需要 Etag？

6. 如何实现一个深拷贝（Deep Clone）？

难度：★★

考点：数据处理、递归

答案要点：

深拷贝需要创建一个完全独立的对象，递归地复制所有层级的属性。

- 基础实现：使用 JSON.parse(JSON.stringify(obj))，但有局限性。
- 递归处理：遍历对象键值，如果是对象则递归调用。
- 循环引用：使用 WeakMap 存储已拷贝的对象，防止死循环。

- 特殊类型：处理 Date、RegExp、Map、Set 等特殊对象的拷贝逻辑。

常见坑：

- 忽略了循环引用导致的栈溢出。
- 丢失了函数、Symbol 或不可枚举属性。

追问：

- 为什么在处理循环引用时建议使用 WeakMap 而不是 Map？

7. Flex 布局中 flex: 1 代表什么意思？

难度：★

考点：CSS 布局细节

答案要点：

flex: 1 是一个简写属性，它决定了元素在主轴方向上如何分配剩余空间。

- 组成部分：它是 flex-grow: 1、flex-shrink: 1 和 flex-basis: 0% 的缩写。
- flex-grow：定义放大比例，1 表示占用所有剩余空间。
- flex-shrink：定义缩小比例，1 表示空间不足时等比缩小。
- flex-basis：定义在分配多余空间之前的基准大小。

常见坑：

- 误以为 flex: 1 的 flex-basis 是 auto。
- 不清楚在 flex-direction 为 column 时它的表现。

追问：

- flex: auto 和 flex: 1 有什么区别？

8. 什么是闭包？它有哪些应用场景和副作用？

难度：★★

考点：作用域、内存管理

答案要点：

闭包是指有权访问另一个函数作用域中变量的函数，本质是函数与其词法环境的引用捆绑。

- 形成条件：函数嵌套且内部函数引用了外部函数的变量。
- 应用场景：封装私有变量、柯里化（Currying）、防抖节流函数。
- 内存影响：闭包会导致外部函数的变量常驻内存，无法被垃圾回收。
- 解决内存泄漏：手动将引用置为 null。

常见坑：

- 在循环中使用 var 定义变量导致闭包取值错误（经典面试题）。
- 过度使用闭包导致性能下降。

追问：

- 现代浏览器引擎是如何优化闭包内存消耗的？

9. 数组去重有哪些实现方式？

难度：★

考点：基础算法、ES6 API

答案要点：

数组去重是常见的数据处理需求，可以利用 ES6 新特性或传统遍历实现。

- Set 方式：Array.from(new Set(arr))，最简洁高效。
- Map/对象键值对：利用 key 的唯一性，适合处理对象数组去重。
- filter + indexOf：通过判断当前索引是否等于首次出现索引。
- 性能对比：Set 和 Map 的时间复杂度通常为 $O(n)$ ，双层循环为 $O(n^2)$ 。

常见坑：

- 无法正确处理 NaN、null、undefined 的去重。
- 无法处理对象引用类型的去重。

追问：

- 如果数组里有数万个元素，哪种方式性能最好？

10. 算法题：将一个扁平的数组转换为树形结构。

难度：★★

考点：数据结构转换、递归/迭代

答案要点：

给定一个包含 id 和 parentId 的对象数组，将其转换为嵌套的树形结构。

- 核心思路：利用 Map 建立 id 到对象的映射，方便快速查找。
- 遍历逻辑：一次遍历，根据 parentId 将当前节点挂载到父节点的 children 数组中。
- 根节点判断：parentId 为空或特定值的作为根节点。
- 空间换时间：使用对象引用特性，避免多次递归查找。

代码块

```
1 function arrayToTree(list) {
2   const map = {}, res = [];
3   list.forEach(item => map[item.id] = { ...item, children: [] });
4   list.forEach(item => {
5     const parent = map[item.parentId];
6     if (parent) parent.children.push(map[item.id]);
7     else res.push(map[item.id]);
8   });
9   return res;
```


常见坑：

- 忽略了原数组中父节点可能在子节点之后出现的情况。
- 未处理存在多个根节点的情况。

追问：

- 如果数据量非常大，递归实现会有什么风险？

二面（深入原理与项目）

1. 谈谈 React Fiber 架构解决了什么问题？它的工作原理是什么？

难度：★★★

考点：框架底层原理、调度机制

答案要点：

Fiber 是 React 16 引入的新的协调引擎，主要解决了大型应用在更新时导致的页面卡顿问题。

- 核心问题：旧版 Stack Reconciler 递归更新不可中断，长期占用主线程导致掉帧。
- 任务拆分：将更新过程拆分为多个小任务（Work In Progress），支持中断与恢复。
- 双缓存技术：在内存中构建新的 Fiber 树，完成后一次性提交到真实 DOM。
- 优先级调度：根据任务紧急程度（如动画 > 数据请求）分配执行时机。

常见坑：

- 混淆了 Render 阶段（可中断）和 Commit 阶段（不可中断）。
- 不清楚 Fiber 节点本质上是一个链表结构。

追问：

- requestIdleCallback 在 Fiber 调度中起到了什么作用？

2. React 中的 Diff 算法是如何优化的？

难度：★★

考点：虚拟 DOM、算法优化

答案要点：

React 的 Diff 算法通过三个前提假设，将 $O(n^3)$ 的复杂度降低到了 $O(n)$ 。

- Tree Diff：只比较同层级节点，跨层级移动会被当作删除和新增。
- Component Diff：相同类组件生成相似树结构，不同类组件直接替换。
- Element Diff：通过 key 属性标识节点，实现节点的复用、移动和删除。

- 策略细节：仅向右移动策略，减少不必要的 DOM 操作。

常见坑：

- 使用数组 index 作为 key 导致的渲染性能问题或状态错误。
- 认为 Diff 算法会找出全局最优解（实际上是局部最优）。

追问：

- 为什么 Vue 的双端 Diff 和 React 的单向 Diff 策略不同？

3. 你们项目里是如何进行性能优化的？有哪些量化指标？

难度：★★★

考点：工程实践、性能监控

答案要点：

性能优化是一个系统工程，需要从网络加载、运行时渲染和资源体积三个维度入手。

- 加载优化：代码分割（Code Splitting）、懒加载、CDN、Gzip、Tree Shaking。
- 渲染优化：虚拟列表（针对长列表）、防抖节流、减少重排重绘、React.memo。
- 量化指标：使用 Lighthouse 或 Web Vitals（LCP, FID, CLS）。
- 监控方案：利用 Performance API 收集数据并上报。

常见坑：

- 只做“感觉上”的优化，没有数据支撑。
- 过度优化，导致代码可读性变差。

追问：

- 针对腾讯文档这种复杂编辑器场景，你会重点优化哪方面的性能？

4. Webpack 的构建流程是怎样的？如何提高构建速度？

难度：★★

考点：工程化、构建工具

答案要点：

Webpack 是一个模块打包器，通过递归构建依赖图谱并调用 Loader/Plugin 处理资源。

- 核心流程：初始化参数 -> 开始编译 -> 确认入口 -> 编译模块（Loader） -> 完成编译 -> 输出资源。
- 速度优化：使用持久化缓存（cache）、多进程打包（thread-loader）、缩小搜索范围（include/exclude）。
- 体积优化：Tree Shaking、代码压缩（Terser）、提取公共第三方库（SplitChunks）。
- 现代方案：对比 Vite 的 ESM 开发模式，解释为什么 Vite 在开发环境更快。

常见坑：

- Loader 和 Plugin 的职责混淆。

- 忽略了 source-map 对构建速度的影响。

追问：

- 什么是 HMR（热更新）？它的实现原理是什么？

5. 谈谈 Canvas 和 SVG 的区别，在文档场景下如何选择？

难度：★★

考点：图形渲染、场景选型

答案要点：

Canvas 适合像素级操作和大量对象的频繁重绘，而 SVG 适合矢量图形和需要 DOM 事件交互的场景。

- Canvas：位图渲染，性能随画布大小变化，适合游戏、复杂动效、大数据量图表。
- SVG：矢量图，基于 XML，支持 CSS 和事件绑定，适合图标、简单地图、文档排版。
- 文档场景：编辑器核心渲染区常选 Canvas（如腾讯文档、飞书文档），以支持海量内容的高性能滚动。
- 交互处理：Canvas 需要手动计算坐标来实现点击交互，而 SVG 可以直接给元素绑事件。

常见坑：

- 在 Canvas 中创建过多对象导致内存溢出。
- 忽略了 Canvas 在高分屏下的模糊问题（需要处理 devicePixelRatio）。

追问：

- 如何解决 Canvas 渲染文字模糊的问题？

6. 协同编辑中的 OT 算法或 CRDT 算法了解吗？

难度：★★★

考点：协同办公、复杂场景处理

答案要点：

协同编辑的核心是解决多个用户同时修改同一份文档时的冲突问题。

- OT (Operational Transformation)：通过操作转换，将本地操作根据远程操作进行偏移修正。
- CRDT (Conflict-free Replicated Data Types)：通过数据结构设计，保证不同副本最终一致，无需中心服务器。
- OT 流程：客户端发送 Op -> 服务器排序并广播 -> 客户端收到后 Transform。
- 适用场景：OT 适合中心化系统（如腾讯文档），CRDT 适合去中心化或 P2P 场景。

常见坑：

- 简单地认为“最后一次修改覆盖”就能解决冲突。
- 不清楚 OT 算法中服务端版本号的作用。

追问：

- 如果网络延迟很高，OT 算法如何保证用户体验？

7. 什么是微前端？你们是如何处理微前端应用间的通信和样式隔离的？

难度：★★

考点：架构设计、微前端

答案要点：

微前端将大型应用拆分为多个独立开发、部署的子应用，实现技术栈无关和增量升级。

- 通信机制：CustomEvent、发布订阅模式、全局状态共享（如 Window 挂载或 Props 传递）。
- 样式隔离：Shadow DOM、CSS Scoping（如 CSS Modules）、运行时动态增删样式表。
- JS 隔离：Proxy 沙箱机制，拦截子应用对 window 的修改。
- 框架选型：qiankun（基于 single-spa）、wujie（基于 iframe + webcomponent）。

常见坑：

- 忽略了全局变量污染导致的子应用崩溃。
- 多个子应用同时存在时的内存泄漏问题。

追问：

- 为什么 qiankun 这种方案比单纯的 iframe 更好？

8. 谈谈前端安全，如何防范 XSS 和 CSRF 攻击？

难度：★★

考点：安全意识

答案要点：

前端安全是保障用户信息和系统稳定的基础，重点在于过滤输入和验证请求来源。

- XSS (跨站脚本攻击)：通过注入恶意脚本窃取 Cookie。防范：转义 HTML、使用 CSP（内容安全策略）、HttpOnly。
- CSRF (跨站请求伪造)：利用用户登录态发起非法请求。防范：使用 Token 验证、SameSite Cookie 属性、验证 Referer。
- 点击劫持：通过 iframe 覆盖。防范：X-Frame-Options 响应头。
- 数据泄露：敏感信息加密传输，避免在前端存储明文密码。

常见坑：

- 认为只要用了 HTTPS 就绝对安全。
- 混淆了 XSS 和 CSRF 的攻击原理。

追问：

- 什么是 CSP？它能预防哪类攻击？

9. 虚拟列表（Virtual List）的实现原理是什么？

难度：★★

考点：长列表优化

答案要点：

虚拟列表通过只渲染可视区域内的 DOM 节点，极大提升了处理海量数据时的渲染性能。

- 核心思路：监听滚动事件，根据 scrollTop 计算当前应显示的起始索引。
- 容器结构：一个固定高度的容器（overflow: auto）和一个撑开高度的占位元素。
- 动态偏移：使用 transform: translateY 移动显示区域，使其始终在视口内。
- 缓冲区：在可视区上下多渲染几个节点，防止滚动过快出现白屏。

常见坑：

- 列表项高度不固定时，计算起始索引变得复杂（需要预估高度或动态测量）。
- 频繁触发滚动事件导致性能抖动。

追问：

- 如果列表项高度是动态的，你会如何优化计算逻辑？

10. 介绍一下你负责过的最难的项目，难点在哪里？你是如何解决的？

难度：★★★★

考点：综合素质、问题解决能力

答案要点：

这是一个开放性问题，重点在于体现你对技术细节的掌控和解决复杂问题的思路。

- 项目背景：简述业务场景和技术栈。
- 核心难点：如性能瓶颈、复杂交互、多端兼容、历史包袱等。
- 解决过程：调研选型 -> 方案对比 -> 灰度验证 -> 最终落地。
- 最终产出：通过数据（如首屏时间降低 50%）或业务反馈证明价值。

常见坑：

- 描述过于笼统，没有具体的技术细节。
- 难点其实不难，体现不出深度。

追问：

- 如果现在让你重新做这个项目，你会做哪些改进？

三面（架构与综合能力）

1. 如果让你从零设计一个前端监控系统，你会考虑哪些模块？

难度：★★★★

考点：系统设计、全链路思维

答案要点：

一个完整的监控系统需要涵盖数据采集、传输、存储、分析和告警五个阶段。

- 采集模块：错误监控（JS/Promise/资源）、性能监控（Web Vitals）、行为监控（埋点/PV/UV）。
- 传输模块：利用 navigator.sendBeacon 保证页面关闭时也能成功上报，支持合并请求。
- 存储与处理：数据清洗、日志存储（如 Elasticsearch）、SourceMap 解析定位源码。
- 展示与告警：可视化看板、阈值告警（邮件/企业微信通知）。

常见坑：

- 忽略了上报本身对业务性能的影响。
- 没考虑 SourceMap 的安全管理。

追问：

- 如何避免监控系统在高并发下的崩溃？

2. 面对一个大型老旧项目，你会如何制定技术升级或重构计划？

难度：★★★

考点：工程领导力、风险控制

答案要点：

重构需要平衡业务发展与技术演进，核心原则是“小步快跑、灰度迭代”。

- 现状评估：梳理核心链路、依赖关系、测试覆盖率和性能痛点。
- 方案选择：是整体重构还是局部优化？是否引入微前端进行渐进式升级？
- 风险控制：建立完善的自动化测试，先在非核心模块试点，实施灰度发布。
- 团队协作：制定代码规范，编写技术文档，确保团队成员理解新架构。

常见坑：

- 盲目追求新技术而忽略了业务稳定性。
- 缺乏灰度方案，上线后出现大面积回滚。

追问：

- 如何说服产品经理和老板给技术重构留出时间？

3. 谈谈你对前端工程化的理解，它包含哪些内容？

难度：★★

考点：工程化视野

答案要点：

前端工程化是为了提高开发效率、保证代码质量和降低维护成本的一系列手段。

- 开发规范：代码风格（ESLint/Prettier）、提交规范（Commitlint）。
- 构建工具：自动化编译、压缩、打包、按需加载。
- 持续集成：CI/CD 流动线，自动化测试、自动部署。

- 提效工具：脚手架、组件库、Mock Server、低代码平台。

常见坑：

- 认为工程化就是配置一下 Webpack。
- 忽略了文档建设和团队规范的软实力。

追问：

- 你们团队目前的 CI/CD 流程是怎样的？

4. 在多人协作的复杂项目中，如何保证代码质量和风格统一？

难度：★★

考点：团队协作、规范执行

答案要点：

通过工具约束和流程管理，将人为因素降到最低。

- 静态检查：强制开启 ESLint 和 TS 强类型约束，并在 Git Hook 中拦截不合规代码。
- Code Review：建立 CR 机制，不仅看逻辑错误，也看设计模式和可维护性。
- 规范文档：沉淀团队的最佳实践手册，减少重复沟通。
- 单元测试：对核心逻辑和公共组件编写单元测试，保证回归稳定性。

常见坑：

- 规范只写在文档里，没有工具强制执行。
- CR 流于形式，只看拼写错误。

追问：

- 如果团队成员对某种技术方案有严重分歧，你作为负责人怎么处理？

5. 跨端技术（RN, Flutter, H5）你如何看待？在什么场景下选哪种？

难度：★★★

考点：技术选型、跨端原理

答案要点：

跨端技术的选择本质是在开发效率、性能表现和动态性之间做权衡。

- H5：开发最快，动态性最强，但性能上限低，适合营销页、简单功能。
- React Native：原生渲染，性能较好，适合追求原生体验且需要 JS 生态的应用。
- Flutter：自绘引擎，性能最接近原生，UI 高度一致，但学习成本和包体积较大。
- 选型逻辑：核心业务追求极致体验选原生；快速迭代选 H5；中大型复杂 App 考虑 RN/Flutter。

常见坑：

- 忽略了跨端方案在特定平台（如折叠屏、低端机）的兼容性坑。
- 没考虑到团队现有的技术储备。

追问：

- 为什么腾讯文档这种产品在移动端会大量使用 Web 渲染而不是纯原生？

6. 你最近在关注哪些前端新技术？它们解决了什么问题？

难度：★★

考点：技术好奇心、学习能力

答案要点：

这是一个考察候选人是否保持技术敏感度的题目，建议结合实际趋势回答。

- 趋势示例：Serverless（降低后端运维门槛）、Rust 在前端工具链的应用（提升构建速度）。
- 框架演进：React Server Components（减少客户端 JS 体积）、Signals 响应式机制。
- 实际应用：比如关注到 Vite 5 的发布，或者 WebAssembly 在处理文档计算逻辑中的应用。
- 思考深度：不仅说出名字，还要说出它背后的原理和对现有方案的改进。

常见坑：

- 只报菜名，说不出具体原理。
- 关注的技术过于陈旧。

追问：

- 你认为 AI（如 Copilot, LLM）会对未来的前端开发模式产生什么冲击？

7. 聊聊你在项目中遇到的最大挫折或失败的经历。

难度：★★★

考点：抗压能力、复盘能力

答案要点：

重点不在于失败本身，而在于你如何面对失败以及从中吸取的教训。

- 场景描述：如一次严重的线上事故、一个延期交付的需求或一个推倒重来的架构。
- 根本原因：是技术预研不足？沟通不畅？还是流程缺失？
- 补救措施：当时是如何快速止损的。
- 复盘总结：事后建立了什么机制来规避同类问题再次发生。

常见坑：

- 甩锅给同事或环境。
- 描述的事情太小，体现不出复盘价值。

追问：

- 经过那次事件后，你在工作习惯上做了哪些改变？

8. 你对腾讯文档前端团队有什么了解？你觉得自己为什么适合这个岗位？

难度：★★

考点：岗位匹配度、职业规划

答案要点：

体现你对业务的思考和自身优势的结合。

- 业务理解：腾讯文档是典型的重交互、重性能、重协同的复杂 Web 应用。
- 优势匹配：如对 Canvas 渲染有经验、深入研究过协同算法、或者有大型复杂项目的架构经验。
- 态度诚恳：表达对挑战复杂技术问题的兴趣，以及对团队技术氛围的认可。
- 职业规划：希望在 Web 编辑器或高性能渲染领域深耕。

常见坑：

- 回答太通用，换个公司也能用。
- 表现出对业务细节完全不感兴趣。

追问：

- 如果入职后让你负责一个非常枯燥的维护任务，你会怎么处理？